LecGap: Diagnosing Comprehension Gaps in Recorded Lectures with Prerequisite Concept Graphs

Soham Gawas
Dept. of CSE (AI & ML)
Bharat College of Engineering
Badlapur, India

Bhargav Ghawali
Dept. of CSE (AI & ML)
Bharat College of Engineering
Badlapur, India

Mahesh Gawali
Dept. of CSE (AI & ML)
Bharat College of Engineering
Badlapur, India

Ayush Devadiga
Dept. of CSE (AI & ML)
Bharat College of Engineering
Badlapur, India

Project Guide: Prof. Swati Gaikwad

***Abstract*—Recorded lectures are now a primary mode of learning, yet they cannot tell a student whether a topic has actually been understood. When a learner fails a question, the cause is frequently a weak prerequisite concept taught earlier, and neither the video nor current AI study tools can locate it. We present LecGap, an end-to-end system that turns a lecture recording into a course-level prerequisite graph and uses it to diagnose and repair comprehension gaps. Lecture audio is transcribed with Whisper; a language model extracts concepts and the passages that teach them; prerequisite edges are taken from explicit statements in the lecture and, for the remaining pairs, from a classifier built on frozen sentence embeddings. A student's wrong answers are traced through the graph to the root-cause prerequisite, and short video clips are served in dependency order. A co-failure rule refines the graph from student responses, and instructors receive a confusion heatmap and a comparison of taught order against learned order. On LectureBank the classifier reaches an F1 score of 0.569, and in simulation the refinement step raises graph F1 from 0.571 to 0.857. The system has processed real lectures of up to 86 minutes and is implemented with FastAPI and Streamlit.**

***Index Terms*—lecture comprehension, prerequisite learning, concept graph, knowledge gap detection, speech recognition, large language models, educational data mining, weak supervision**

I. Introduction

Recorded lectures from platforms such as NPTEL and YouTube have made high-quality teaching widely available, and students routinely pause, rewind and revisit them. However, a video can only be replayed; it cannot report whether the viewer has understood it. In technical subjects the difficulty is compounded by dependency: a learner cannot follow backpropagation without gradient descent, nor gradient descent without loss functions. When a quiz answer is wrong, the real cause is often a missing prerequisite taught earlier, and neither the learner nor the video makes that dependency visible.

Two bodies of work address parts of this problem. Prerequisite relation learning infers which concepts must be understood before others [1], [2], but is typically trained offline on fixed benchmarks and never corrected by learners. Knowledge tracing models student mastery from interaction history [3], [4], but treats skills as given and has no access to the content of a particular lecture. Commercial AI study tools summarise recordings and generate quizzes, yet score each question independently. None connects a wrong answer to the structure of the lecture in which the concept was taught, and instructors receive no view of class-wide confusion.

This paper presents **LecGap**, a system that closes this gap. LecGap builds a prerequisite graph directly from lecture audio, links quiz results to that graph, recommends short clips in dependency order, and refines the graph from student behaviour. Our contributions are:

1) An end-to-end pipeline from raw lecture video to a timestamped, course-level concept graph, including robust handling of long recordings and malformed model output.
2) A transcript-first prerequisite strategy: dependencies stated by the instructor are stored as evidence edges, and a lightweight classifier on frozen sentence embeddings covers the remaining pairs.
3) A root-cause remediation method that traces wrong answers through the graph and orders short clips by prerequisite dependency.
4) A co-failure refinement rule that corrects graph edges from student responses, validated with simulated students.
5) An open implementation with background jobs, live progress, a student dashboard, and a faculty dashboard showing a confusion heatmap and taught-versus-learned divergence.

The remainder of the paper is organised as follows. Section II surveys related work, Section III describes the system, Section IV its implementation, Section V the experiments, and Section VI discusses limitations and future work.

II. Related Work

A. Prerequisite Relation Learning

Li et al. introduced LectureBank, a corpus of lecture slides with 208 topics annotated with prerequisite relations, and framed prerequisite chain learning as pairwise classification [1]. Pan et al. studied concept prerequisites in MOOCs using features derived from course content and embeddings [2], and Liang et al. proposed measuring prerequisite relations from how concepts refer to one another [5]. Li et al. later showed that prerequisite structure can be transferred across domains without target-domain labels, using variational graph autoencoders [6]. These works establish that prerequisites are learnable, but they take the concepts as given, are evaluated offline, and use no feedback from learners. LecGap differs by extracting concepts from raw lecture audio and by correcting its graph from student responses.

B. Knowledge Tracing

Bayesian Knowledge Tracing models mastery of a skill as a hidden state updated after each response [3]. Deep Knowledge

Tracing replaces it with a recurrent network that predicts the next response from the full interaction history [4]. Both predict whether a student will answer correctly, but they do not use the lecture content and cannot say which earlier concept to revisit.

C. Foundations for Lecture Processing

Whisper, trained on large-scale weakly supervised audio, provides robust timestamped transcripts [7]. CLIP learns joint image–text representations and, combined with OCR, can describe slide content [8]. Sentence-BERT yields sentence embeddings that can be compared by cosine similarity [9]. NetworkX supplies directed-graph operations, including cycle detection and topological sorting [10]. LecGap composes these components rather than replacing them.

D. AI Study Tools

Commercial tools such as YouLearn, Knowt and NotebookLM summarise recordings and generate flashcards or quizzes. They operate on text rather than course structure, grade each answer independently, and offer no class-level view.

E. Summary

Table I compares these approaches on the capabilities that matter for lecture comprehension. No surveyed approach combines a lecture-derived graph with root-cause remediation, learning from students and instructor analytics.

TABLE I
COMPARISON OF RELATED APPROACHES

Approach	**Concept graph**	**Root-cause fix**	**Learns from students**	**Instructor analytics**
Prerequisite classifiers [1], [2]	Yes (offline)	No	No	No
Cross-domain transfer [6]	Yes (offline)	No	No	No
Knowledge tracing [3], [4]	No	No	Yes	No
AI study tools	No	No	No	No
LecGap (proposed)	Yes (per course)	Yes	Yes	Yes

III. PROPOSED SYSTEM

A. Overview

Fig. 1 shows the pipeline. A lecture recording is transcribed, concepts and the passages that teach them are extracted, and prerequisite edges are inferred and merged into a directed acyclic graph (DAG). Short clips are cut for each concept, a quiz is generated, and a student's wrong answers are traced through the graph. Responses also feed graph refinement and the instructor dashboard.

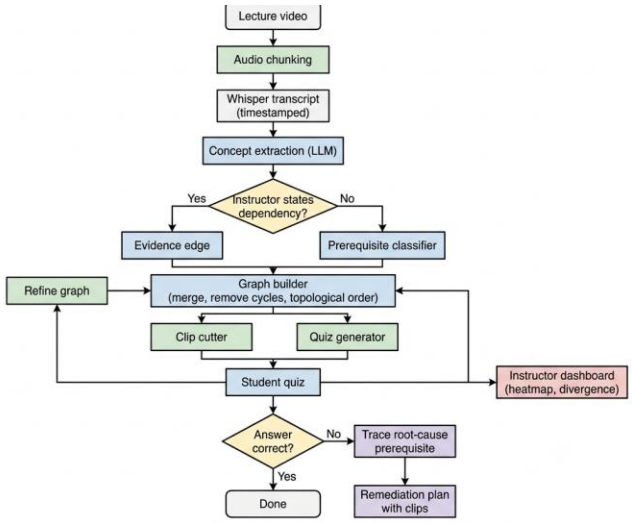


Fig. 1. End-to-end LecGap pipeline from lecture video to remediation plan.

B. Transcription and Concept Extraction

Recordings are split into audio chunks of at most five minutes and re-encoded to avoid malformed tails. Each chunk is transcribed with Whisper [7] through a hosted service with retry and back-off, and segment timestamps are shifted back onto the lecture timeline. Rather than treating the transcript as unrelated chunks, a lecture-structure pass reads overlapping windows of segments and asks a language model to return the passages in which each concept is taught, the concept names, and the spoken links between concepts (for example, "this builds on gradient descent"). Truncated or invalid JSON output is repaired or salvaged so that complete passages are never lost. Every concept is anchored to the time span in which it is taught, which later drives clip cutting, the lecture timeline and quiz generation.

C. Prerequisite Inference

Edges come from two sources. Spoken links are stored as evidence edges with high confidence together with the supporting sentence. For the remaining pairs, a classifier decides whether concept A is a prerequisite of concept B. Concept names are embedded with a compact Sentence-BERT-style encoder (MiniLM) [9]; the pair feature vector concatenates the two embeddings with their absolute difference, their element-wise product and their cosine similarity, and a logistic-regression classifier [11] is trained on LectureBank with negative undersampling. The decision threshold is chosen on a held-out validation slice so that the test split is never used for tuning. Cross-encoder fine-tuning was also evaluated (Section V-B) and did not improve on this baseline.

D. Graph Construction

Concept names are embedded and near-duplicates are merged into canonical nodes, with a map from raw to canonical names. The merge threshold is deliberately conservative, because wrongly merging distinct concepts is more harmful than keeping near-duplicates. When an edge is proposed more than once, the highest confidence is kept, and self-loops created by

merging are dropped. Cycles are removed by repeatedly finding a cycle and deleting its lowest-confidence edge until the graph is acyclic, and a topological sort gives a valid learning order [10].

E. Quiz and Root-Cause Remediation

For each concept, a multiple-choice question is built from the lecture's own sentences, with distractors drawn from other concepts of the same lecture, and the quiz length is capped. Answer keys remain on the server and grading is performed there. For each wrong concept, the system follows its incoming prerequisite edges and checks the student's answers on those nodes. Concepts that the student also failed, or that are the earliest unmet prerequisites, are placed first in the remediation plan, and the clip for each concept is attached in that order. For example, a student who fails backpropagation and also gradient descent is directed first to gradient descent and then to backpropagation.

F. Graph Refinement

An offline-trained prerequisite model cannot adapt, so LecGap refines its graph from responses using a co-failure rule over many students (Fig. 2). If students who fail concept *A* also tend to fail concept *B*, the edge from *A* to *B* is reinforced; if they fail *A* but not *B*, it is weakened. Over many responses, spurious edges lose confidence and genuine edges gain it. Refinement runs after batches of responses and never removes a transcript-derived edge without evidence from several students, so a single unusual student cannot damage the graph.

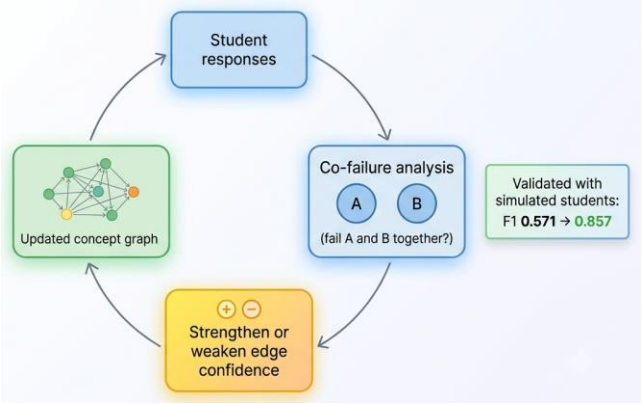


Fig. 2. Co-failure refinement loop. Simulated validation: graph F1 0.571 to 0.857.

G. Instructor Analytics

Aggregated responses are shown to instructors as a confusion heatmap over the lecture timeline. The order in which concepts were taught is also compared with the order given by the graph's topological sort, which highlights lectures that teach a concept before its prerequisites.

IV. Implementation

A. Architecture

LecGap has four layers (Fig. 3). Two Streamlit dashboards, one for students and one for faculty, form the presentation layer and talk to the back end through a thin HTTP client. A FastAPI application [12] provides the service layer: routes for courses, lectures, graphs, clips, quizzes and statistics, plus background jobs. The pipeline layer holds the processing modules for transcription, structure extraction, prerequisite classification, graph construction, clip cutting, quiz generation and refinement. The data layer stores structured data in a relational database and media and clips on disk.

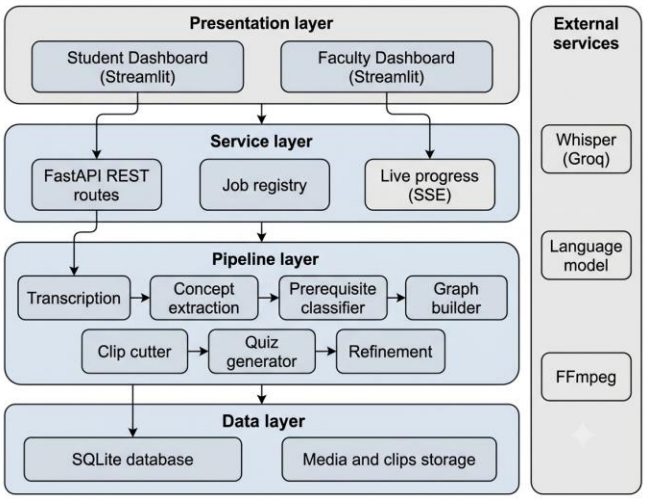


Fig. 3. Layered system architecture of LecGap.

B. Jobs and Reliability

Long operations never block the interface. Each is registered as a durable job with a status, a stage and a progress percentage, and the back end streams changes to the front end using server-sent events, with polling as a fallback. Language-model responses are cached so repeated runs are fast. Clips are cut with FFmpeg in parallel. Uploads are size-limited and file names are sanitised, answer keys are never sent to the client, and access can be restricted with an API key. The code base has more than 700 automated tests covering the back end and the dashboards.

V. Experiments and Results

A. Data

Prerequisite classification uses LectureBank [1]: 208 labelled concepts with 921 positive prerequisite edges among 42,749 concept pairs. End-to-end experiments use NPTEL and YouTube lectures on machine learning, deep learning and probability, from 21 to 86 minutes long.

B. Prerequisite Classification

The frozen-encoder classifier achieves a cross-validated F1 of 0.569 (Table II). Fine-tuned cross-encoders, evaluated over grids of epochs, learning rates and negative ratios on a GPU and also with a larger MPNet backbone, scored between 0.49 and 0.55, all below the baseline. We therefore applied a shipping gate, adopting a fine-tuned model only if it clearly beats the baseline, and kept the frozen-encoder classifier.

TABLE II
PREREQUISITE CLASSIFICATION ON LECTUREBANK

Model	Cross-validated F1
Frozen MiniLM + logistic regression (adopted)	0.569
Fine-tuned cross-encoders (various settings)	0.49 – 0.55

C. Graph Refinement

Refinement was validated with simulated students (Table III). With 200 structural personas the graph F1 rose from 0.571 to 0.857 with a precision of 1.0, and with eight language-model personas that genuinely attempted the quiz it rose from 0.571 to 0.750. These are simulated results: they show that the mechanism can recover a correct graph under controlled conditions and are not a claim about live classrooms.

TABLE III
GRAPH REFINEMENT WITH SIMULATED STUDENTS

Simulated students	F1 before	F1 after
200 structural personas	0.571	0.857
8 language-model personas	0.571	0.750

D. System Behaviour

Hosted Whisper transcribes at about ten times real-time speed. The pipeline completed end-to-end runs on lectures between 21 and 86 minutes long. Chunked audio, JSON salvage, retry with back-off and durable jobs were each added in response to failures observed on long recordings.

VI. DISCUSSION AND LIMITATIONS

Two lessons stand out. First, explicit statements by the instructor are a stronger source of prerequisites than a name-only classifier, which is why LecGap stores them as evidence edges and uses the classifier only as a fallback. Second, a simpler model that is benchmarked honestly is preferable to a more complex one that does not beat it.

The work has limitations. The refinement results use simulated students and need confirmation in a live classroom. LectureBank covers a limited number of topics, mainly in natural language processing, so transfer to other subjects may be imperfect. Concepts shown only on slides are not yet captured because the visual track is still under development. Transcription and language-model steps depend on external services, and the baseline has been run on a single machine without load testing. No user study has yet been conducted.

VII. CONCLUSION AND FUTURE WORK

We presented LecGap, a system that turns a lecture recording into a prerequisite graph, traces a student's wrong answers to root-cause prerequisites, serves short clips in dependency order, and refines the graph from student responses while giving instructors a class-level view. A frozen-encoder classifier reached an F1 of 0.569 on LectureBank, and simulated refinement raised graph F1 from 0.571 to 0.857.

Future work includes a React front end, completing the visual concept track with CLIP and OCR, hardening the job queue for multi-user deployment, testing cross-domain transfer, and, most importantly, evaluating the system with real students to measure learning outcomes.

ACKNOWLEDGMENT

The authors thank their project guide and the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Bharat College of Engineering, Badlapur, for their guidance and support.

REFERENCES

[1] I. Li, A. R. Fabbri, R. R. Tung and D. R. Radev, "What should I learn first: Introducing LectureBank for NLP education and prerequisite chain learning," in *Proc. AAAI Conf. Artificial Intelligence*, 2019.

[2] L. Pan, C. Li, J. Li and J. Tang, "Prerequisite relation learning for concepts in MOOCs," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2017.

[3] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Model. User-Adapted Interact.*, vol. 4, no. 4, pp. 253–278, 1994.

[4] C. Piech *et al.*, "Deep knowledge tracing," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015.

[5] C. Liang, Z. Wu, W. Huang and C. L. Giles, "Measuring prerequisite relations among concepts," in *Proc. Conf. Empirical Methods Natural Language Process. (EMNLP)*, 2015.

[6] I. Li, V. Yan and D. Radev, "Unsupervised cross-domain prerequisite chain learning using variational graph autoencoders," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2021.

[7] A. Radford *et al.*, "Robust speech recognition via large-scale weak supervision," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023.

[8] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Language Process. (EMNLP-IJCNLP)*, 2019.

[10] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008.

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[12] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com.